Reproducing and Extending LLM-as-a-Judge: MT-Bench, FastChat, and New Evaluation Data

Uday Reddy Poli
Department of Computing, Macquarie University
COMP8240: Applications of Data Science, Session 2 2026
`Project Final Report, Assessment Task #3`

Abstract

This report describes the completion of a semester-long project reproducing and extending Zheng et al.'s LLM-as-a-judge paper, *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena* [Zheng et al., 2023], using the authors' open-source FastChat codebase. The original paper's headline claim – that GPT-4 agrees with human preference judgments more than 80% of the time, comparably to human-human agreement – was reproduced directly from the authors' own released vote data, obtaining 85.9% (turn 1) and 85.3% (turn 2) GPT-4-vs-human agreement, excluding ties. The original codebase was installed and run successfully, surfacing and fixing two real, previously undocumented issues: missing transitive dependencies in the package's declared install extras, and a category-registration bug that crashes grading on any new question category. Sixteen new MT-Bench-style questions were constructed across two categories – harder coding problems and Australian local-context knowledge – with unit-tested reference answers for all eight coding questions, and were evaluated with an LLM-as-judge pass using Claude as a documented, openly-stated substitute for the original paper's GPT-4 judge, since no OpenAI API access was available. The substitute judge produced a mean score of 8.13/10 across the 16 new questions, with real score variation (6–9) indicating the judge is discriminating between answer quality rather than defaulting to uniform scores.

1 Description of Paper, and Justification

Zheng et al.'s *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena* [Zheng et al., 2023] (NeurIPS 2023 Datasets and Benchmarks track) addresses a practical problem in evaluating large language models: human preference annotation is the gold standard for judging open-ended chat quality, but it is slow and expensive to collect at scale. The paper proposes and validates *LLM-as-a-judge*: using a strong model (GPT-4) to score or compare model responses in place of a human annotator. Its evaluation methodology compares GPT-4's judgments against a large pool of crowdsourced human preference votes, on MT-Bench (their own constructed multi-turn benchmark) and on Chatbot Arena conversations. On MT-Bench, the paper reports that GPT-4 agrees with human majority-vote preference more than 80% of the time, a level of agreement comparable to (and, excluding ties, exceeding) the agreement rate between two independent human annotators.

The code for this project is lm-sys's FastChat repository on GitHub, specifically its `llm_judge` module.[1] FastChat was chosen for three reasons. First, feasibility was verified upfront rather than assumed: the repository is public, actively maintained, and well-documented, and because the judging step is pure API calls rather than local model inference, it does not require a GPU – the only real requirement is an OpenAI API key and a modest amount of paid credit. Second, the paper is highly influential: it was published at NeurIPS 2023 (a CORE A* venue)

[1] https://github.com/lm-sys/FastChat

and has accumulated over 600 citations on Google Scholar as of this project. Third, the topic – LLM-as-a-judge and model evaluation methodology – sits squarely in an area directly relevant to applied data science practice, where it is increasingly common to use one model to evaluate another's outputs cheaply.

2 Description of the Original Dataset

The paper's benchmark is MT-Bench: 80 carefully designed multi-turn questions spanning eight categories – writing, roleplay, extraction, reasoning, math, coding, STEM, and humanities/social science – with ten questions per category. Each question has exactly two turns, so a model being evaluated must hold context across a follow-up instruction rather than answer a single isolated prompt. For two categories, math and coding, the dataset also ships a reference answer per question, used to ground the judge's grading in those categories (FastChat's `NEED_REF_CATS` list). Figure 1 shows a verified snippet confirming the dataset loads correctly.

```
$ python -c "import json; data=[
  json.loads(l) for l in
  open('data/mt_bench/question.jsonl')];
  print(len(data),'questions');
  print(sorted(set(
    d['category'] for d in data)))"
80 questions
['coding', 'extraction', 'humanities',
 'math', 'reasoning', 'roleplay',
 'stem', 'writing']
```

Figure 1: Verified output confirming all 80 MT-Bench questions and 8 categories load correctly from the original dataset file.

The dataset is distributed directly inside the FastChat repository (`fastchat/llm_judge/data/mt_bench/question.jsonl`) in a simple JSON-lines schema: each line has a `question_id`, a `category`, and a `turns` list of the two prompt strings. Reference answers, where present, follow the same per-question-id JSON-lines format in a separate `reference_answer.jsonl` file. The authors additionally released the original GPT-4 judge votes and 3.3K human preference votes used to produce the paper's headline agreement figure, as a public dataset on Hugging Face (`lmsys/mt_bench_human_judgments`),[2] in exactly the vote-record format FastChat's own `compute_agreement.py` script expects – this proved essential for Section 3 below.

3 Replication of Original Work

3.1 Environment and running the original code

The original pipeline has two stages. `gen_api_answer.py` calls each candidate model's API and saves its answers to the 80 MT-Bench questions. `gen_judgment.py` then sends those answers – plus the question, and for math/coding a reference answer – to GPT-4, which returns a numeric judgment per answer. The environment was set up and verified hands-on, twice: once in a cloud working environment, and independently again on a personal MacBook Air (Python 3.12.7, venv), to rule out an environment-specific fluke. Both times, FastChat was cloned, its `llm_judge` dependencies installed, and all 80 questions across all 8 categories were confirmed to load and the scripts to execute without errors (Figure 1). All project scripts, patches, and test files described in this report are available in the project's public GitHub repository.[3]

3.2 Two real issues found and fixed

Installing and actually importing the judging code (not just reading it) surfaced two genuine, previously undocumented problems, beyond the two already-known install quirks from the proposal stage (a `wavedrom` build failure, and an `openai<1` version pin needed for API compatibility).

Undeclared transitive dependencies. The package's own declared `llm_judge` install extras (in `pyproject.toml`) list only `openai<1`, `anthropic>=0.3`, and `ray`. Actually importing `fastchat/llm_judge/common.py` end-to-end surfaced three more hard require-

[2] https://huggingface.co/datasets/lmsys/mt_bench_human_judgments

[3] https://github.com/REPLACE_WITH_USERNAME/comp8240-llm-judge-project – **see note to self in the submitted README: replace this footnote URL with the actual repo link before final PDF compile.**

ments not visible from the declared extras: `anthropic` is imported unconditionally even when not used as a judge, and, transitively, `torch`, `transformers`, and `accelerate` are required because `common.py` unconditionally imports `fastchat.model.model_adapter` – a module that is actually only needed for *local* model inference, not pure API-based judging. All four were installed and the import now succeeds cleanly.

Category-registration bug. Adding a new question category and attempting to grade it raises `KeyError: 'au_context'` from the `temperature_config` dictionary in `common.py`, because that dictionary has no default or fallback entry – every category used for grading must be registered by name in advance. This was fixed with a one-line patch (`"au_context": 0.7,`, matching the temperature already used for the similarly open-ended `writing`/`roleplay` categories, since `au_context` questions are not reference-checked). The fix was verified: the KeyError no longer occurs, and the category receives a sensible non-deterministic sampling temperature.

3.3 Headline result, reproduced

Running a full live GPT-4 judging pass over all 80 original questions was not required to reproduce the paper's central claim, because the authors released their own original GPT-4 votes and 3.3K human votes as a public dataset in exactly the format their own agreement script expects (Section 2). This made it possible to reproduce the headline result directly and for free, with zero live API spend, using `reproduce_headline_agreement.py`, which loads that dataset and computes both GPT-4-vs-human and human-vs-human agreement, with and without ties, mirroring the original script's logic. (Hugging Face is blocked by the cloud sandbox's network policy, so this script was instead run on a personal machine.) Table 1 shows the result.

Turn	GPT-4 vs human	human vs human
1	85.9% (911/1060)	82.0% (633/772)
2	85.3% (912/1069)	83.4% (649/778)

Table 1: Reproduced agreement rates, excluding ties, from the authors' own released vote data.

Both turns reproduce the paper's "$>80\%$" headline claim, and in both turns GPT-4's agreement with human preference is actually slightly *higher* than human-vs-human agreement itself, excluding ties – consistent with the paper's own finding that GPT-4 agreement is comparable to, or exceeds, the level of agreement between two independent human annotators. This is the strongest piece of evidence for replication in this project: a real number, independently reproduced from the authors' own data, not merely a claim that the code "should work."

4 Construction of New Data

Two categories of new data were constructed, both following FastChat's exact two-turn question schema (`question_id`, `category`, `turns`), so they drop directly into the existing pipeline without any format conversion.

Eight harder coding questions (ids 1001–1008): linked-list cycle detection, bounded-memory top-k frequency counting over a log file, a buggy binary search requiring a fix, a SQL `DENSE_RANK` window-function query, a race condition and its fix, a thread-safe token-bucket rate limiter, bracket/tag balance checking, and closest-pair-of-points via the haversine formula. These were chosen to be more varied and harder than the original benchmark's coding questions, and because `coding` is in FastChat's `NEED_REF_CATS` list, each needs a reference answer to be judged in reference-guided mode.

Eight Australian local-context questions (ids 2001–2008): small-business grants, GST vs payroll tax, local government structure, preferential voting, the Medicare Safety Net, buying a first home, visa pathways, and daylight saving – a kind of local, time-sensitive factual knowledge the original benchmark never covers.

4.1 Reference answers and their testing

Real, correct reference solutions were written for all 8 coding questions, and **all 8 are unit-tested** in `test_refs.py` – executed against concrete inputs, not eyeballed, including a 100-thread concurrency stress test for the rate limiter (confirming the lock caps allowed requests at exactly the bucket's capacity despite 100 simultaneous callers) and a temp-file-backed test for the IP-counting question.

Running `python test_refs.py` prints `ALL 8 REFERENCE ANSWERS VERIFIED` on a clean run. Figure 2 shows a representative snippet of this.

```
$ python test_refs.py
1003 OK
1007 part1 OK
1007 part2 OK
1001 OK
1004 OK
1002 part1 OK
1002 part2 OK
1005 OK
1006 OK
1008 OK
ALL 8 REFERENCE ANSWERS VERIFIED
```

Figure 2: Output of the reference-answer test suite: all 8 coding reference answers pass.

One question's wording was corrected during this testing process. Question 1002's turn 1 originally asked for a solution using "bounded memory," but the reference solution's own memory use is $O(\text{number of distinct IPs})$, not a fixed bound – the turn-1 solution's own documentation already acknowledged this. The question now asks to solve it "without loading the entire file into memory," which is what the turn-1 solution actually achieves; the genuinely fixed-memory, approximate version (a count-min-sketch-style structure) is introduced in turn 2 instead. This makes the two-turn progression internally accurate: turn 1 is exact but distinct-IP-bounded, turn 2 is approximate but truly fixed-memory.

4.2 Annotation and quality control

Rather than treat the 16 questions as self-evidently correct, each was reviewed against an explicit criterion before inclusion, recorded in full in `annotation_record.md`. Coding questions were reviewed against *executable correctness*: does the reference answer, run against concrete test inputs, produce the behaviour the question asks for? All 8 passed this review (post-fix). The Australian-context questions were reviewed against a different criterion, chosen deliberately because these questions touch facts that change over time (grant amounts, thresholds, visa categories): not "is there one fixed correct answer," but "does the question push the model toward a defensible, well-framed answer, with appropriate uncertainty where figures could have changed." This criterion was carried forward into the judging pass in Section 5.

No crowdsourced human annotation was used for this new data – all construction and review was carried out by a single annotator (the author), which is itself discussed as a limitation in Section 6, since it means no inter-annotator agreement measure exists for the new questions, unlike the original paper's large crowdsourced human-vote pool.

5 Results on New Data

Evaluating the 16 new questions with the original pipeline requires an LLM judge, but no OpenAI API key was available for a live GPT-4 run through `gen_api_answer.py` / `gen_judgment.py`. Rather than report a pure limitation with no results, this stage used Claude (via the assistant session supporting this project) as a documented, free substitute for *both* roles the original pipeline needs: generating one independent set of 16 candidate answers (`candidate_answer_additions.jsonl`, written without reference to the reference answers), and then, in a separate judging pass, scoring those candidate answers on FastChat's 1–10 single-answer grading scale – reference-guided against `reference_answer_additions.jsonl` for the coding category, and against the factual-correctness-plus-appropriate-uncertainty criterion from Section 4.2 for the Australian-context category. Full scores and per-question notes are in `judge_results.md`.

This substitution is a stated methodology deviation, not a hidden one. Using the same underlying model family for both the candidate and the judge role carries a known self-evaluation bias risk discussed in the LLM-as-judge literature, including in the original paper itself, and Claude is not GPT-4, the specific model the original paper evaluates. The results below should therefore be read as evidence that the new questions and their grading criteria function correctly end-to-end, not as a reproduction of what a live GPT-4 judging run would score.

Scores ranged from 6 to 9 across the 32 individual (question, turn) gradings – real variation, not a uniform ceiling score, which is useful evidence

Category	Turn 1	Turn 2	Overall
Coding (n=8)	8.50	7.88	8.19
Au-context (n=8)	8.13	8.00	8.06
All (n=16)	8.31	7.94	**8.13**

Table 2: Mean judge scores (1–10 scale) on the 16 new questions.

that the grading criteria are actually discriminating between stronger and weaker answers. The single lowest-scoring answer (1002, turn 2, score 6) was a candidate answer whose eviction-based memory-bounding strategy was functionally adequate but noticeably cruder than the reference's count-min-sketch approach – exactly the kind of difference a reference-guided judge should catch.

What this does *not* show is GPT-4-vs-human agreement specifically on this new data, since no human annotation pass or live GPT-4 run was performed on these 16 questions, unlike the original 80-question result reproduced in Section 3 from the authors' own released human votes. Running a live `gen_api_answer.py` / `gen_judgment.py` pass against GPT-4 on this new data, budget permitting, remains the natural next step and is discussed further in Section 7.

6 Ethical Considerations

In the original paper. The original paper's evaluation relies on a large pool of crowdsourced human preference annotators. This raises standard crowdsourcing ethics questions the paper itself discusses only briefly: annotator compensation and working conditions, the demographic composition of the annotator pool relative to the population whose preferences are being modelled, and the risk that preference judgments reflect superficial qualities (length, confident tone, formatting) rather than genuine correctness or helpfulness – a bias the LLM-as-judge literature has since documented further. Using a single commercial model (GPT-4) as judge also concentrates evaluation power in one proprietary system controlled by one company, raising fairness and access concerns: researchers without budget for GPT-4 API access cannot fully replicate or audit the judging step, which is itself directly illustrated by this project's own budget constraint in Section 5.

For this project's new data. If the Australian-context questions and their model evaluations were to be published as a research dataset, under the Australian Code for the Responsible Conduct of Research [Australian Code, 2018], the most relevant considerations are: (1) *data currency and potential for misinformation* – several questions touch rules that change over time (grant thresholds, visa categories, Medicare Safety Net amounts), so any published dataset or model trained/evaluated against it would need a clear "as-of" date and explicit framing that answers should not be treated as current legal or financial advice; (2) *single-annotator bias* – all 16 questions, their review criteria, and (in this report) the substitute judge's scores were produced by one person/model pipeline, with no inter-annotator agreement measure, unlike the original paper's large human-vote pool; publishing this data as a benchmark would require either a second independent annotator or explicit disclosure of this limitation; (3) *no personal or sensitive data is involved* – all 16 questions concern public government policy information or general programming problems, not individuals, so there are no participant-consent or privacy concerns of the kind the Code's human-research provisions address; and (4) *transparency about the substitute judge*, consistent with the Code's research-integrity principles around honest reporting of methods and limitations – which is why Section 5 states the GPT-4-to-Claude substitution explicitly rather than presenting the scores as equivalent to the original paper's judging methodology.

7 Conclusions, and Other Reflections

This project set out to replicate Zheng et al.'s LLM-as-a-judge paper and extend it to new data. Replication succeeded on every piece attempted: the original codebase installs and runs cleanly (after fixing two real, previously undocumented issues), and the paper's headline >80% GPT-4-vs-human agreement claim was reproduced directly from the authors' own released vote data (85.9% and 85.3% across the two turns, excluding ties), exceeding human-vs-human agreement itself. Extension to new data was also completed: 16 new two-turn questions were con-

structed across two categories not covered by the original benchmark, with all 8 coding reference answers written and unit-tested, and a documented, openly-stated substitute LLM-as-judge pass was run across all 16 questions, producing real, non-uniform scores (mean 8.13/10, range 6–9) that demonstrate the questions and grading criteria work end-to-end.

The main limitation, acknowledged throughout rather than hidden, is budget: no OpenAI API access was available, so neither the original 80 questions nor the 16 new questions could be run through a live GPT-4 judging pass directly. The 80-question result was still reproduced exactly, because the authors generously released their own vote data; the 16-question result could not be reproduced the same way (it is new data with no existing released votes), so a substitute judge was used instead, with its limitations stated plainly in Section 5. If given further time and budget, the clearest next step would be running `gen_api_answer.py` and `gen_judgment.py` against GPT-4 directly on the 16 new questions, and ideally collecting a small human-preference pass (even just 2–3 annotators) on a subset, to get a genuine agreement-rate figure for the new data comparable to Table 1's figure for the original 80 questions. A second reflection is on annotation process: constructing the Australian-context questions surfaced how much harder it is to write a good open-ended factual question with a *stable* grading criterion than to write one with a single fixed answer – several early draft questions (not included here) asked for specific dollar figures that would have gone stale within a year, which is what motivated the "factual correctness of stable structure plus appropriate uncertainty" criterion used throughout Section 4.2, rather than exact-answer matching.

References

[Zheng et al., 2023] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. https://github.com/lm-sys/FastChat

[Chen et al., 2021] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

[Australian Code, 2018] National Health and Medical Research Council, Australian Research Council, and Universities Australia. 2018. *Australian Code for the Responsible Conduct of Research*, 2018 edition. Canberra: National Health and Medical Research Council.